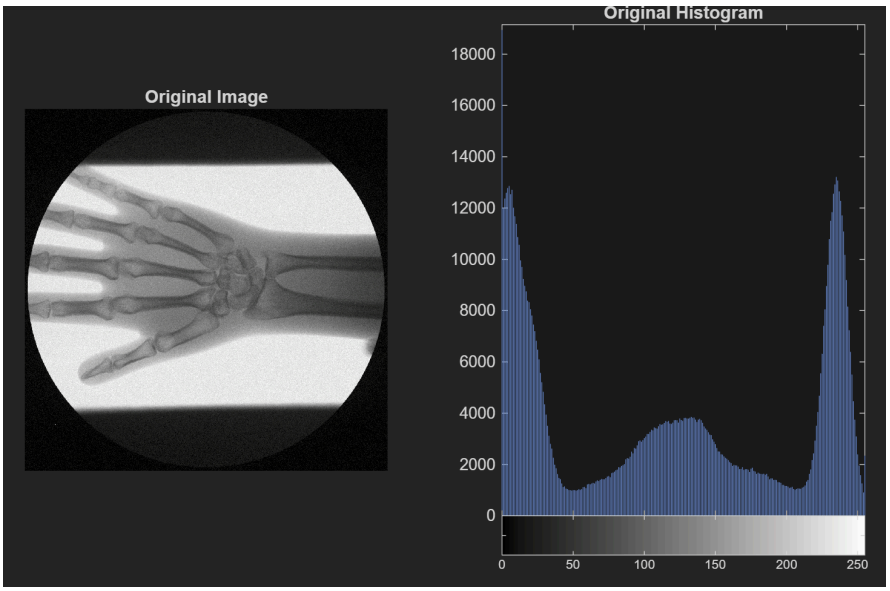
Report Homework 2

Exercise 1

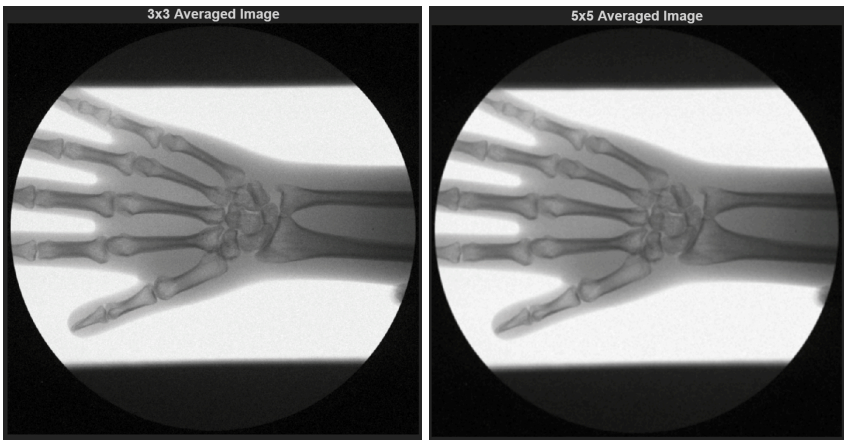
Image loading and pre-processing

After loading the image "hand.mat" and displaying both the original image and its histogram to assess noise levels, Gaussian noise was clearly present.



To remove this noise, the image was pre-processed using various averaging filters (with 3x3, 5x5, 7x7, and 9x9 kernels) and Gaussian filters (σ=1,1.5,2 and 3).

Visual inspection revealed that the 5x5 averaging filter provided the optimal balance, as the 3x3 kernel left residual noise while larger 7x7 and 9x9 kernels caused excessive blurring.







For Gaussian filtering, σ=2 proved superior, since lower values (σ=1,1.5) retained too much noise, leading to noisy edge detection, while σ=2 yielded cleaner, more stable edges ideal for subsequent processing.



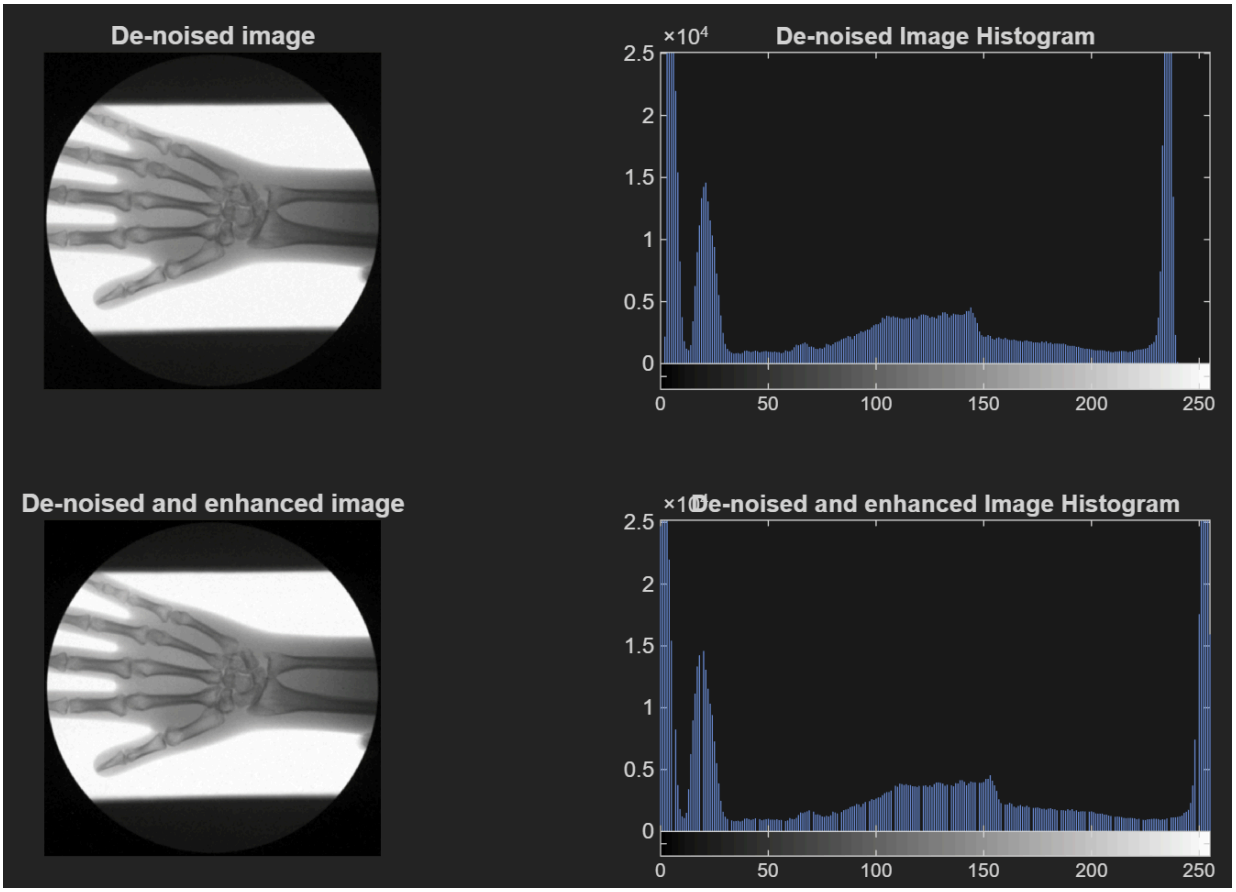






Multiple pre-processing methods were tested to systematically evaluate their impact on downstream segmentation performance, ensuring the selection of the variant that maximized edge quality and overall robustness. Ultimately, the σ=2 filtered image was used for segmentation with compass masks, while for the second method using Canny edge detection, the σ=3 pre-processed image was selected as it produced less noisy segmentation results.

Additionally, histogram stretching was applied to further enhance contrast using stretchlim to exclude the lowest and highest 1% of pixel intensities, followed by imadjust to remap the dynamic range. This step served to expand the image's limited contrast range, making edges more distinguishable for segmentation by utilizing the full grayscale spectrum , as per standard histogram equalization principles.



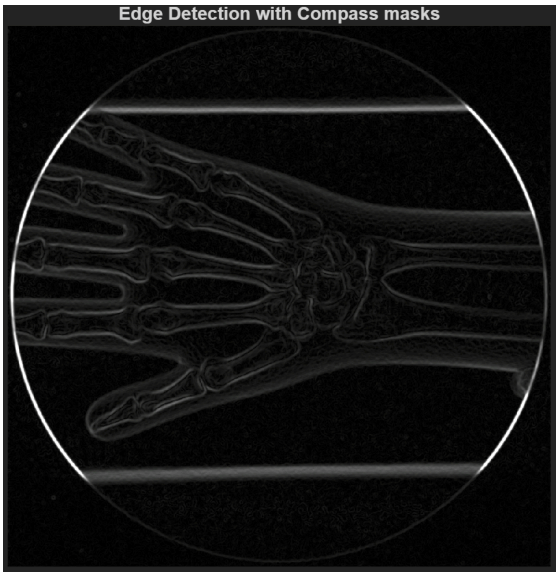
1) Edge detection with Compass masks

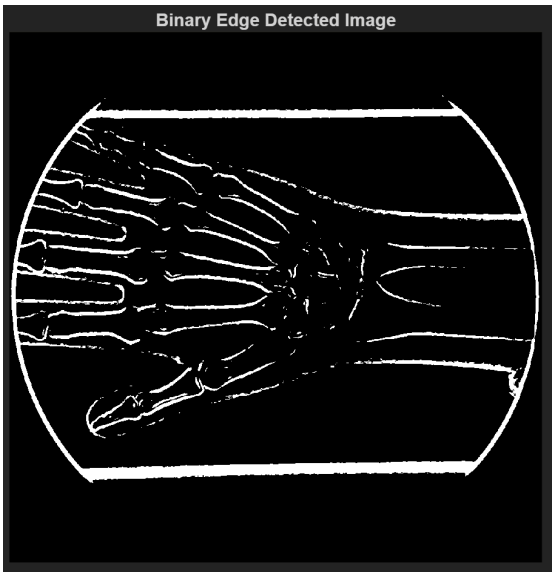
Sobel masks were defined for all 8 principal directions to perform comprehensive edge detection. The standard horizontal Sobel mask was generated using fspecial("sobel"), while the 45° mask was manually defined as [2, 1, 0; 1, 0, -1; 0, -1, -2]. The remaining masks (90°, 135°, 180°, 225°, 270°, 315°) were obtained through successive 90° rotations using rot90().

Each mask was convolved with the denoised and contrast-enhanced image (img_contrast) using imfilter(..., 'conv', 'replicate') to detect edges in its respective direction. This produced 8 separate edge maps (edges_sobel, edges_sobel45, ..., edges_sobel315), where positive convolution values indicate edge strength and orientation.

A 3D stack was created with cat(3, ...) containing all 8 edge maps along the third dimension. For each pixel, the maximum value across all 8 directions was selected using [edges_max, dir_idx] = max(edges_stack, [], 3), yielding a single edge strength image (edges_max) and a direction index map (dir_idx). This approach ensures that every edge pixel is captured regardless of its orientation, producing a complete edge map without directional bias.

The final edge-detected image was displayed using imshow(edges_max, []) with the title "Edge Detection with Compass masks".





The edge-detected image was thresholded to create a binary edge map. A threshold value was computed as threshold = 0.15 * max(edges_max(:)), representing 15% of the maximum edge strength. The binary image was generated with edges_binary = edges_max > threshold, retaining only pixels exceeding this threshold.

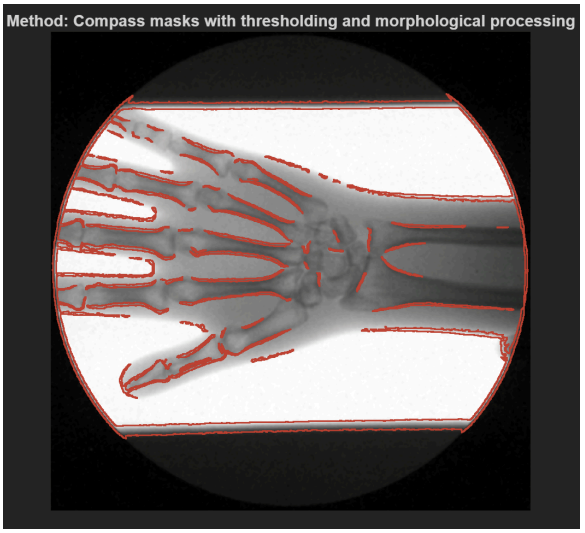
The threshold was selected empirically through iterative visual inspection. Multiple values were tested to identify the optimal balance that removed maximum background noise while preserving bone edge detection, ensuring clean contours without losing critical anatomical structures.

Results were visualized in two ways. First, the binary edge map was displayed standalone using imshow(edges_binary, []) with title "Binary Edge Detected Image". Second, the contrast-enhanced original image (img_contrast) was shown with red contours overlaid via contour(edges_binary, 'LineColor', 'r', 'LineWidth', 1), titled "Method: Compass Masks with thresholding". This superposition clearly demonstrates the accuracy of bone edge localization for segmentation.



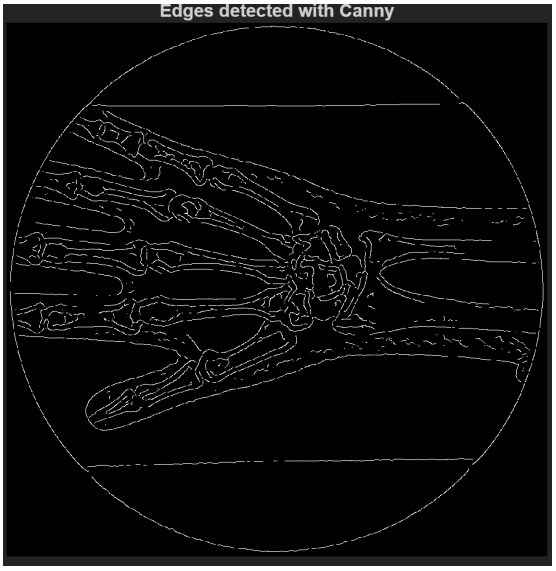
Small noise regions were removed using area-based filtering. The bwareaopen(edges_binary, minArea, 8) function with minArea = 40 eliminated all connected components (8-connectivity) smaller than 40 pixels from the binary edge map, producing edges_clean. This step suppressed isolated noise pixels and tiny artifacts, although the edge detected image continued to appear very noisy and the edges of the bones weren't completely defined.

Perimeter pixels were extracted and overlaid for visualization. The bwperim(edges_clean, 8) function identified boundary pixels of the cleaned edge regions using 8-connectivity. These perimeter coordinates were obtained via [r, c] = find(perimeters) and plotted as red dots (plot(c, r, 'r.', 'MarkerSize', 2)) over the contrast-enhanced image (img_contrast).



2) Edge detection with Canny

Canny edge detection was applied as the second segmentation approach to the Gaussian-filtered image with σ=3 (img_gauss3) using edge(..., 'Canny'). This stronger denoising (σ=3) was specifically chosen over σ=2 because it produced cleaner, less noisy contours critical for accurate bone segmentation.



The Canny method's multi-stage processing, gradient computation, non-maximum suppression, and hysteresis thresholding, proved particularly effective for this noisy hand X-ray, yielding thin, continuous bone edges with minimal artifacts. The resulting binary edge map confirms its suitability as a robust alternative to compass masks for the second segmentation pipeline.

Morphological cleaning was applied with an increased minimum area threshold of minArea = 50 using bwareaopen(edges_canny, minArea, 8). This empirically optimized value removed connected components smaller than 50 pixels (8-connectivity), suppressing noise artifacts while maintaining bone edge integrity.

Perimeter extraction was performed consistently with the first method using bwperim(edges_canny_clean, 8) to isolate boundary pixels. These coordinates ([r, c] = find(perimeters)) were plotted as red dots (plot(c, r, 'r.', 'MarkerSize', 2)) over img_gauss3, titled "Method: Canny with morphological processing".

This unified visualization approach across both methods highlights clean, continuous bone perimeters, validating the Canny pipeline's effectiveness for segmentation after stronger Gaussian denoising (σ=3).



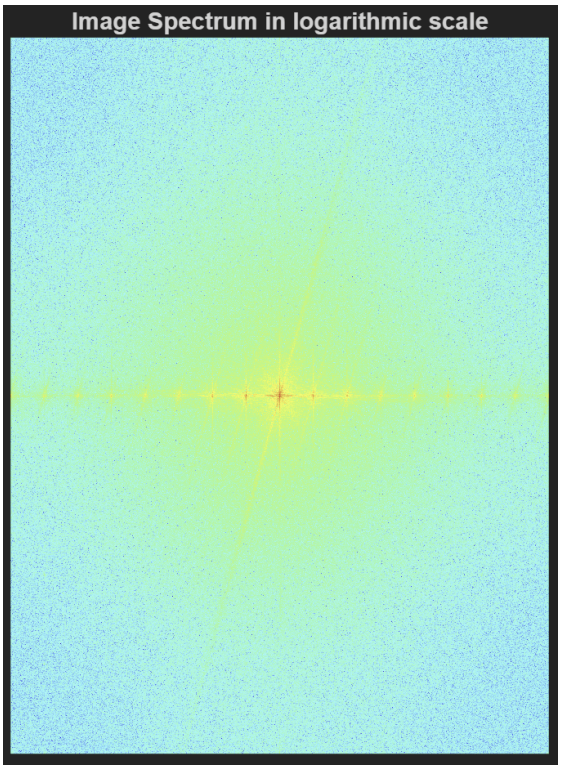
Exercise 2

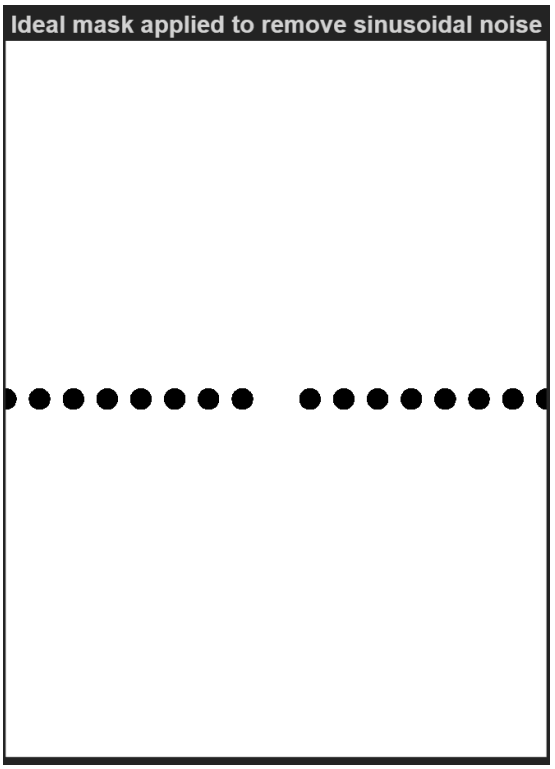
1) Removal of sinusoidal noise



Sinusoidal noise was removed using frequency-domain filtering with an ideal notch mask. The approach leverages the Fourier Transform to identify and suppress specific periodic noise frequencies visible as symmetric peaks in the image spectrum, while preserving the underlying image content.

The image was converted to grayscale double precision, then transformed via fft2() and fftshift() to center the DC component. The logarithmic spectrum (log(1+img_spectrum_shifted)) revealed 16 equally spaced peaks along the horizontal axis (8 per side, spacing = 186 pixels from center at (1489, 1985)). Spatial frequencies were computed as f_x = |x_peak - center_x| * (1/C) cycles/pixel, confirming periodic sinusoidal interference.





A binary mask (mask = ones(R, C)) was constructed with circular notches of radius 60 pixels centered at each noise peak location. For each of the 8 left-side peaks, symmetric notches were placed at (centerx±186i, center_y) for i=1 to 8 using distance grids [u, v] = meshgrid(1:C, 1:R). The radius was empirically optimized to attenuate noise peaks without damaging adjacent frequencies.

The mask multiplied the centered spectrum (F_filtered = img_spectrum_shifted .* mask), followed by inverse operations ifftshift() and ifft2() to recover the spatial domain image (img_filtered_line). This produced a clean image free of sinusoidal patterns, visualized with title "Filtered image with ideal notch filter".



An averaging filter was applied to suppress residual noise remaining after notch filtering using a 19×19 kernel (kernel19 = ones(19,19) * 1/361) normalized to sum to 1. The convolution imfilter(img_filtered_line, kernel19, "same", "conv") produced img_filtered_avg, smoothing the image while maintaining its dimensions.

This step was essential because the ideal notch filter effectively removed dominant sinusoidal components but left subtle residual noise that would interfere with subsequent edge detection required for segmentation. The large kernel size was chosen to provide uniform smoothing across the image, ensuring clean input for the next processing stages. The result, titled "Filtered and smoothed image", confirms effective noise reduction while preserving overall image structure.



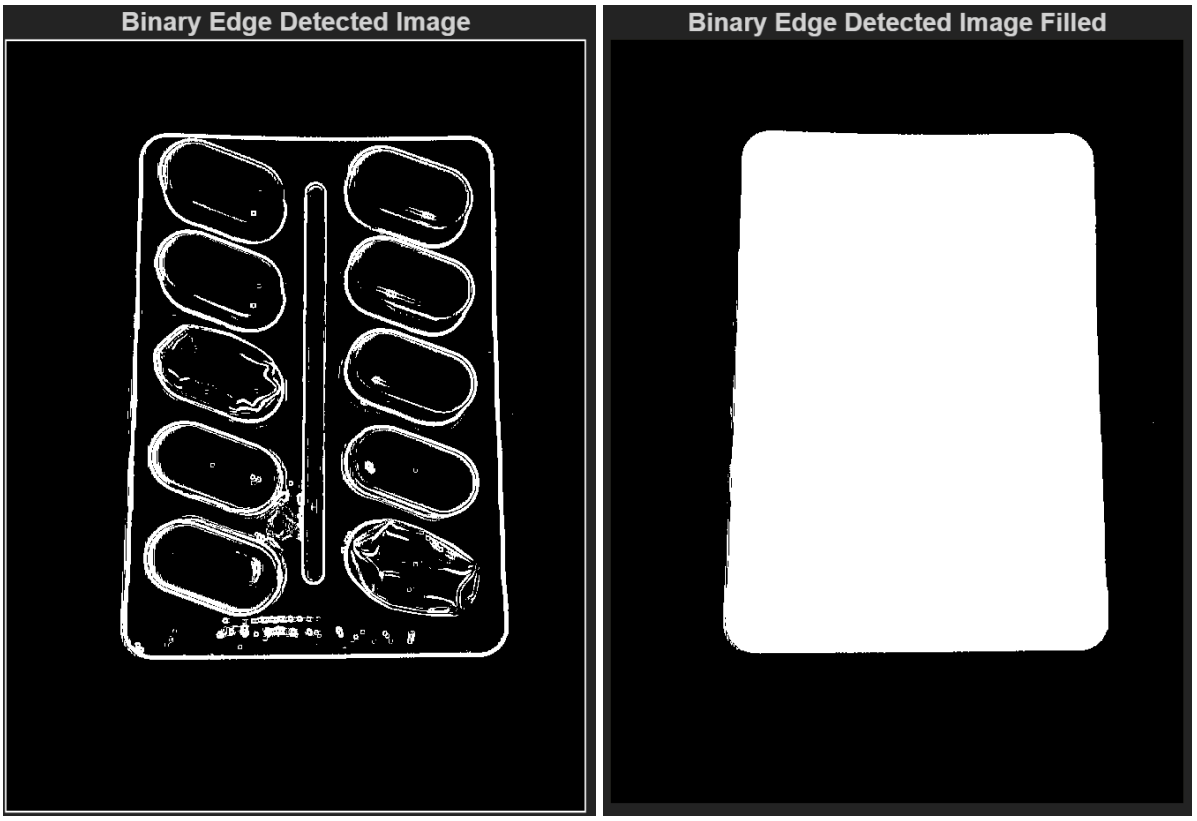
2) External contours and aspect ratio computation

Compass masks were selected for edge detection after comparative testing proved superior to more precise methods like Canny, which produced significant errors likely due to residual sinusoidal noise. Eight Sobel masks covering principal directions were defined (fspecial("sobel") plus rotations), convolved with the smoothed image (img_filtered_avg), stacked in 3D (cat(3, ...)), and reduced to maximum edge strength per pixel (max(edges_stack, [], 3)).

Binary edge map was thresholded and refined. A threshold of 15% maximum edge magnitude (0.15 * max(edges_max(:))), optimized through trials, created edges_binary. The map was cropped 12 pixels from borders (edges_binary(12:end-10, 12:end-10)) to remove artificial contours, then holes filled (imfill(..., "holes")) yielding edges_filled.

Aspect ratio was computed from minimal bounding rectangle dimensions. Height: sum(any(edges_filled, 2)) counts rows containing at least one non-zero value, yielding the height

value of the blister. Width: sum(any(edges_filled, 1)) similarly counts columns with at least one non-zero value, 2 columns were subtracted to account for external noise pixels. Final ratio: cols_with_one / rows_with_one, printed as Aspect ratio: %.4f. The resulting aspect ratio is: 0.7379.



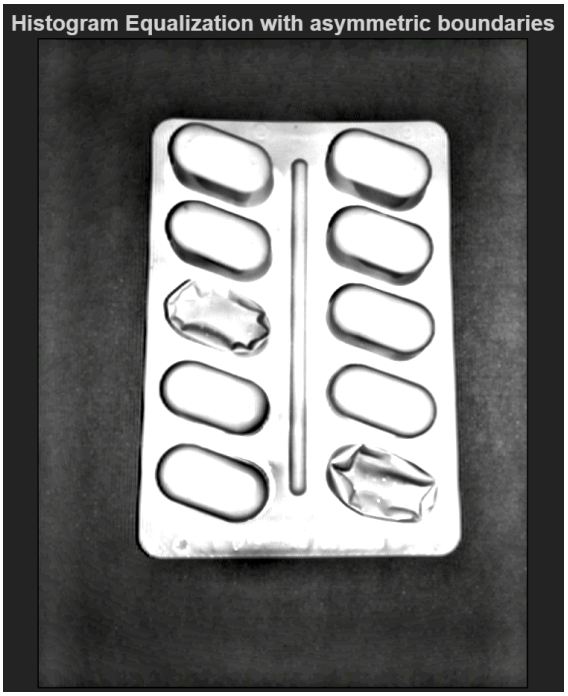
3) Removal of illumination effects

Adaptive histogram equalization corrected local lighting variations. The blister image showed uneven illumination (darker center, brighter edges). adapthisteq(..., 'NumTiles', [16 16]) divided the image into 256 local 64×64 regions, equalizing contrast independently in each tile. 'ClipLimit', 0.01 prevented noise amplification by clipping the top 1% of local histogram values, producing img_adapthist with uniform local contrast.

Saturation capped bright regions at 0.55, empirically optimized for segmentation. The threshold img_saturation(img_adapthist > 0.55) = 0.55 was selected after multiple trials, evaluating which value enabled the best subsequent edge detection and blister segmentation. This prevented bright backgrounds from compressing mid-tone blister details during equalization.

Asymmetric stretching maximized blister visibility. stretchlim(..., [0.01 0.99]) excluded 1% outlier pixels at both ends, then imadjust(..., [0 0.55]) mapped this range to [0, 0.55]. The upper limit matched saturation, concentrating contrast where the blister appears (mid-tones), yielding img_histeq optimal for edge detection.

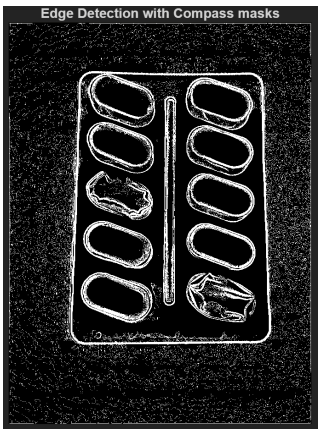


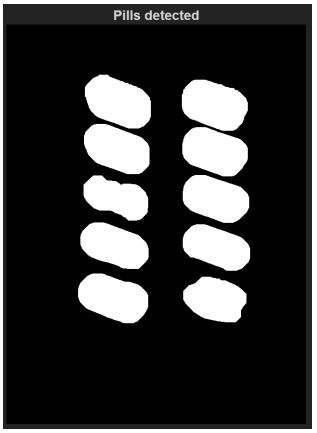


4) Detect the pills still in the blister and the missing ones

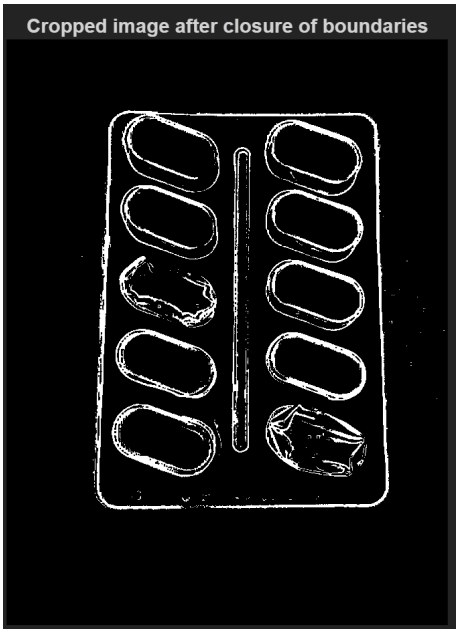
The strategy I chose for this point in the exercise used dual thresholds on compass mask edges to distinguish all pills from unused ones only. A lower threshold (10% of max edge strength) captured faint edges from both used and unused pills, while a higher threshold (15%) detected only the strong, clear edges of unused pills. Subtracting these masks identified used pill locations, with extensive morphological cleanup ensuring accurate counting.

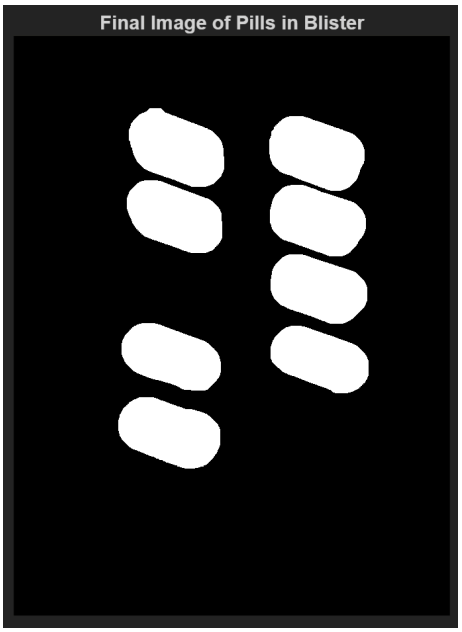
The lower-threshold map (edges_binary_adapt1) underwent bwareaopen(..., 400) to remove noise and imclose('disk',5) to bridge gaps. Cropping used the blister contour from edges_filled ([i_blister, j_blister] = find(edges_filled == 1)) (used to compute the aspect ratio of the blister) to compute bounding box [min_row, max_row, min_col, max_col]. A 15-pixel margin inside the blister was zeroed: top (1:min_row+14), bottom (max_row-14:end), left (1:min_col+14), right (max_col-14:end). This adaptive cropping prevented imfill('holes') from filling the entire blister—only pill regions were completed. Final imopen('disk',100) cleaned artifacts, and bwlabel(...,8) counted numTOT.



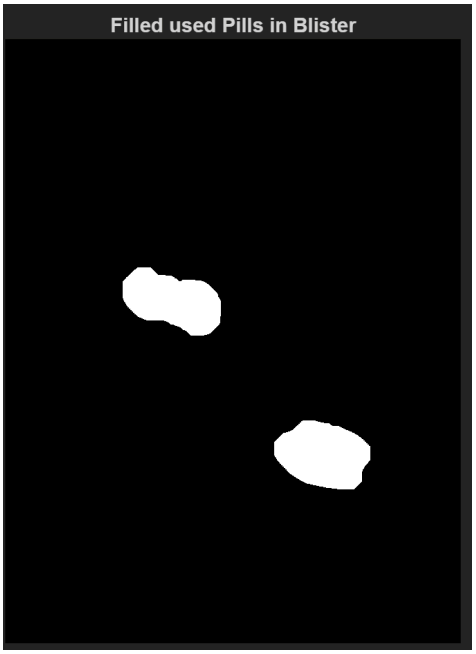


The higher threshold threshold2 = 0.15 * max(edges_max(:)) isolated strong unused pill edges. These were skeletonized (bwmorph('thin')), gaps closed (imclose('disk',5)), borders zeroed, holes filled, and cleaned (imopen('disk',80)). bwlabel counted unused pills as numUNUSED.





Used pills were computed by subtracting unused mask from total mask (edges_filled_pills - edges_filled_pills_unused), with negatives set to zero and final cleanup (imopen('disk',9)). The original image was overlaid with green dots for unused pills and red dots for used pills (plot(..., '.g') and plot(..., '.r')), with counts printed: total, unused, and used (numTOT - numUNUSED). All thresholds and morphological parameters were empirically tuned for optimal pill separation.





Exercise 3

General pipeline

The pipeline begins with thorough data preparation, essential to ensure robust and unbiased model training. Both datasets are initially split into training and testing sets, preserving the original class distribution to prevent bias. Given that medical datasets often suffer from class imbalance, where some classes have significantly more samples than others, methods such as stratified splitting are employed to maintain representative ratios in both sets. Additionally, feature scaling is applied to the data, standardizing the input features so that all have comparable scales. This step is particularly important for algorithms sensitive to feature magnitudes, such as Support Vector Machines or k-Nearest Neighbors.

Next, hyperparameter optimization is conducted via GridSearchCV, using 5-fold cross-validation to further prevent overfitting and assess generalization. Each model has a tailored grid of hyperparameters reflecting its unique characteristics, and the accuracy metric guides the search for the optimal parameter combinations.

After training, models are evaluated on the held-out test set. Accuracy provides an overall performance measure, while detailed classification reports offer insights into precision, recall, and F1-score per class.

Finally, the pipeline includes visualization of confusion matrices to facilitate interpretability. This comprehensive approach not only compares multiple algorithms on two complex datasets but also ensures that preprocessing, balancing, and validation techniques are rigorously applied.

Tested models

The models tested for each dataset are the following:

- K-nearest Neighbors
- SVM
- Logistic Regression
- Decision tree
- Random Forest
- LDA

Model	**Best model (DS1)**	**Accuracy (DS1)**	**Best model (DS2)**	**Accuracy (DS2)**
KNN	Best hyperparameters (Ultrasound): {'metric': 'manhattan', 'n_neighbors': 5, 'weights': 'distance'}	0.98	Best hyperparameters (Brain): {'metric': 'manhattan', 'n_neighbors': 11, 'weights': 'distance'}	0.65
SVM	Best hyperparameters (Ultrasound - SVM):	0.99	Best hyperparameters (Brain - SVM): {'C':	0.67

	{'C': 10, 'gamma': 'scale', 'kernel': 'poly'}		10, 'gamma': 'scale', 'kernel': 'rbf'}	
Logistic Regression	Best hyperparameters (Ultrasound - Logistic Regression): {'C': 10, 'penalty': 'l1', 'solver': 'liblinear'}	0.98	Best hyperparameters (Brain - Logistic Regression): {'C': 10, 'penalty': 'l1', 'solver': 'liblinear'}	0.59
Decision Tree	Best hyperparameters (Ultrasound - Decision Tree): {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}	0.88	Best hyperparameters (Brain - Decision Tree): {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2}	0.50
Random Forest	Best hyperparameters (Ultrasound - Random Forest): {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 300}	0.98	Best hyperparameters (Brain - Random Forest): {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 4, 'min_samples_split': 2, 'n_estimators': 200}	0.64
LDA	Best hyperparameters (Ultrasound - LDA): {'n_components': 1, 'shrinkage': 0.5, 'solver': 'lsqr'}	0.98	Best hyperparameters (Brain - LDA): {'shrinkage': 0.5, 'solver': 'lsqr'}	0.60

Discussion of results

Across all five classifiers, Dataset 1 (Ultrasound) consistently outperformed Dataset 2 (Brain), with accuracies ranging from 88-99% versus 50-67% respectively. SVM achieved the highest performance on both datasets (accuracies 99% and 67% respectively). While the worst model for both datasets is Decision tree, with accuracies equal to 88% and 50% respectively.

This significant performance gap can be attributed to several key factors. First, Dataset 1 contains substantially more samples (981 images per class after balancing the dataset to avoid bias, 2943 total) compared to Dataset 2 (590 images per class, 1770 total), which has approximately 40% fewer samples. This reduced sample size limits the models' ability to learn robust discriminative patterns, particularly for complex multiclass problems.

Second, visual inspection reveals markedly higher intra-dataset similarity in Brain images. Scrolling through Dataset 2 images compared to Dataset 1, images from different classes appear substantially

more similar. This elevated inter-class similarity fundamentally reduces class separability, making accurate discrimination significantly more challenging than in Ultrasound images.

Exercise 4

Initial Processing of datasets

The ultrasound dataset is slightly larger, containing 3,659 images in total. After splitting the data, I allocated 60% for training (2,196 images), 20% for validation (731 images), and the remaining 20% for testing (732 images). The brain dataset follows the same distribution pattern but with fewer images overall: 2,948 total images split into 1,769 training, 589 validation, and 590 test images.

All images were resized to 224×224 pixels, which is the standard input size expected by the pre-trained models I selected (EfficientNetB0 and ResNet50). I used Keras's built-in dataset loading functions to organize the images directly from their directory structure, which automatically inferred the class labels based on folder names.

One important practical consideration was batch size. While larger batches can speed up training, they also require more memory. Given the computational resources available, I reduced the batch size from 64 to 32, which helped prevent memory issues while still maintaining reasonable training speeds.

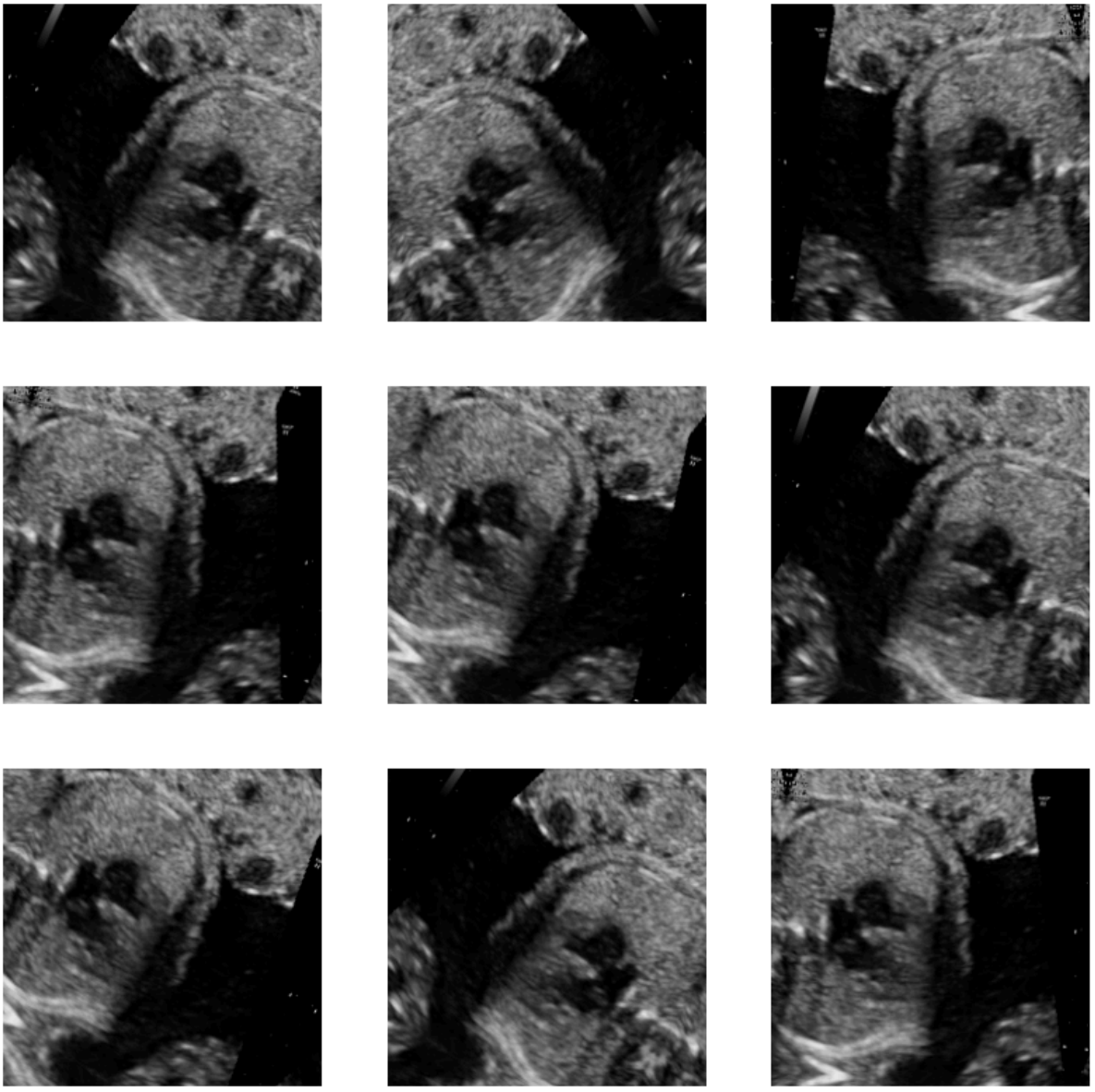
1) Data augmentation

The training sets were augmented using the following transformations applied on-the-fly (only on the training set, leaving the validation and test set untouched):

- RandomFlip (horizontal)
- RandomRotation (±10%)

These augmentations were implemented through Keras preprocessing layers mapped to the training pipeline. Validation and test sets remained unaugmented to maintain consistent evaluation conditions.

Here’s an example of data augmentation from Dataset 1



2) Model training and fine-tuning

Architecture Selection:

Two pre-trained architectures were selected for comparison:

- **EfficientNetB0**: explained in class.
- **ResNet50**: this model was chosen for practical reasons, in particular the inputs required by this architecture are identical to the ones required by the first model

Both models were initialized with ImageNet weights, with the top classification layers frozen.

Training Pipeline:

The training followed a standard two-phase transfer learning approach:

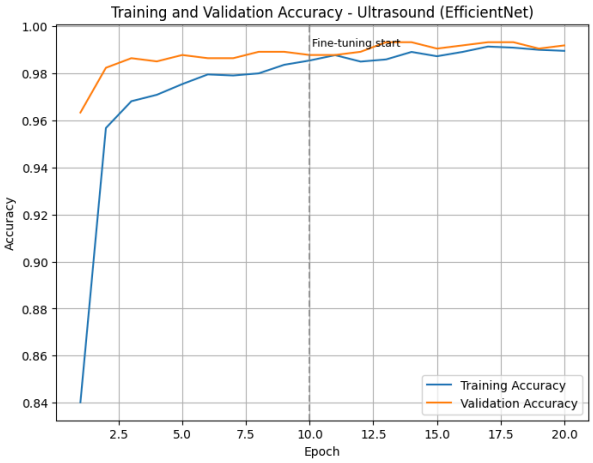
Phase 1 (10 epochs): Feature extraction with frozen base layers. The classification head consisted of GlobalAveragePooling2D, Dense(512, ReLU), Dropout(0.3), and Dense(3, Softmax). Adam optimizer with default learning rate (0.001) and categorical crossentropy loss.

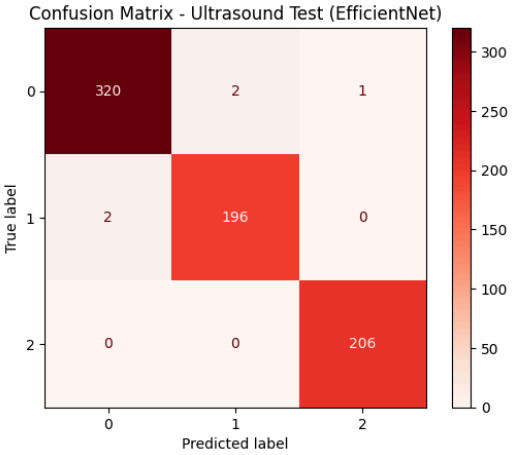
Phase 2 (10 epochs): Fine-tuning with the last 20 layers unfrozen and learning rate reduced to 1e-5 to prevent forgetting of pre-trained features.

3) Evaluation and comparison of results

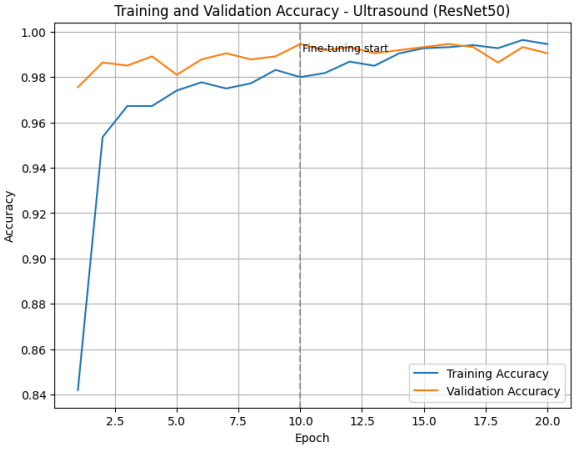
Dataset 1:

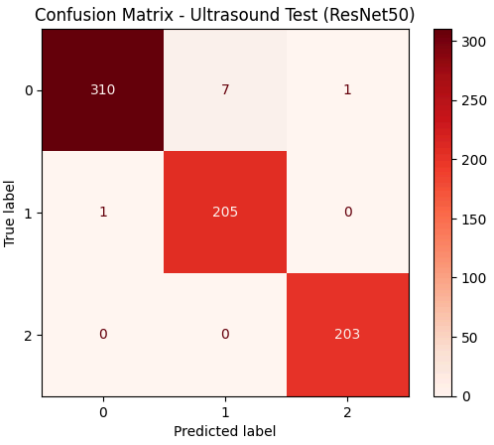
EfficientNetB0 achieved the best performance with 99.04% test accuracy. Training and validation accuracies converged around 98-99%, indicating minimal overfitting. The confusion matrix showed near-perfect classification with negligible off-diagonal elements.





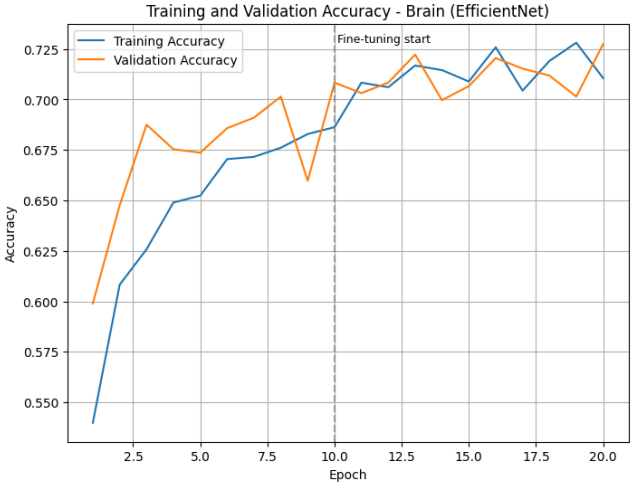
ResNet50 slightly underperformed with 98.8% test accuracy, with similar training and validation accuracies (98-99%).

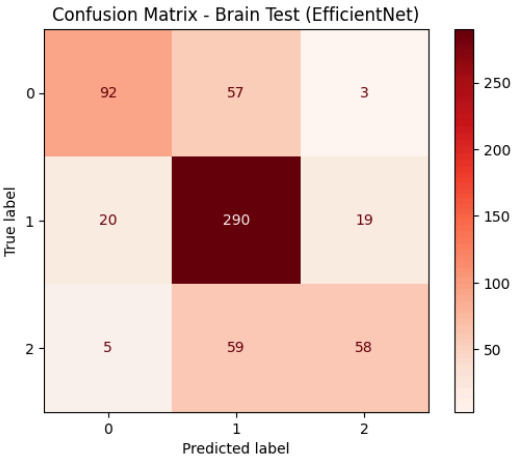




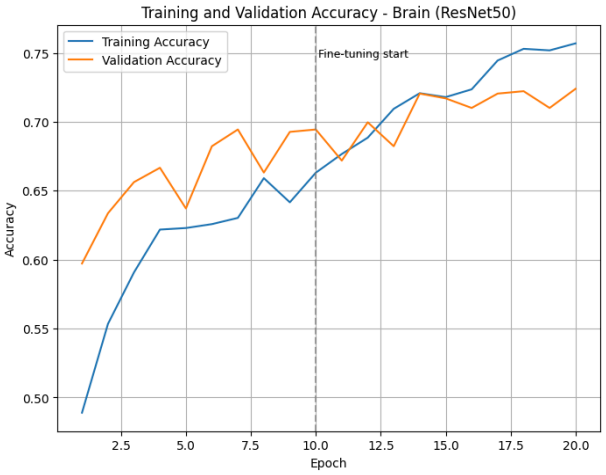
Dataset 2:

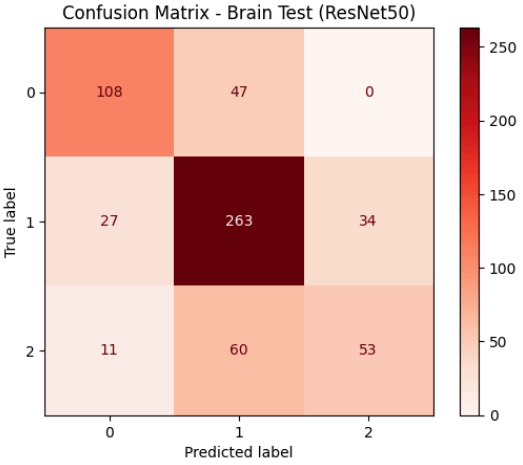
EfficientNetB0 achieved 70.98 % accuracy on the test set and slightly better results on the training and validation sets (respectively 73% and 72%)





ResNet50 achieved 70.32% accuracy on the test set, while on the training and validation sets achieved 75 and 72%





The ~30 percentage point accuracy drop from Dataset 1 to Dataset 2 across both architectures indicates that dataset characteristics dominate model selection in determining final performance. The ultrasound planes exhibit more distinctive visual features, while brain plane classification presents inherent ambiguity that limits achievable accuracy regardless of architecture choice.

Transfer learning proved highly effective for both datasets despite the domain shift from natural to medical images. EfficientNetB0 emerged as the superior choice, offering better accuracy on simpler tasks while maintaining competitive performance on complex ones, with additional benefits of reduced computational requirements.